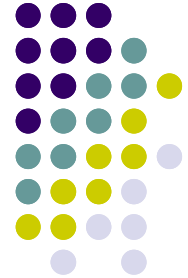


Flyweight



Architecture of Computer Games
SE 456

Ed Keenan

11 February 2019

13.0.6.4.3 Mayan Long Count



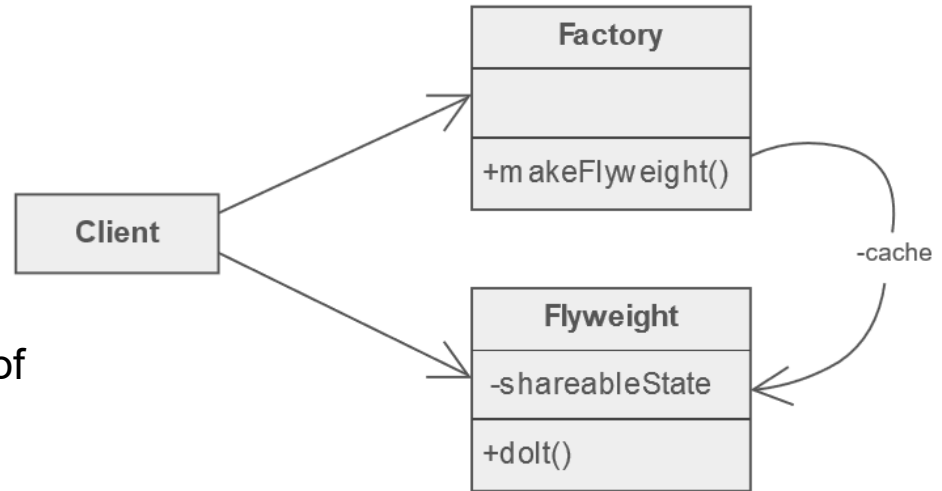
Flyweight

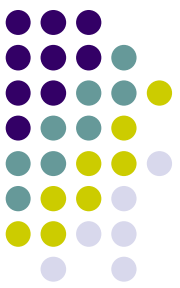
- Intent

- Use sharing to support large numbers of fine-grained objects efficiently.

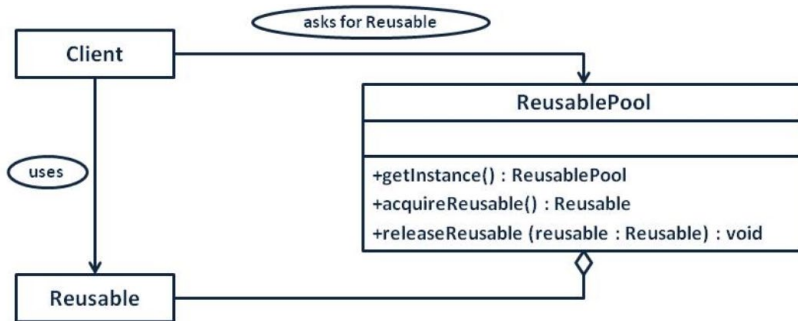
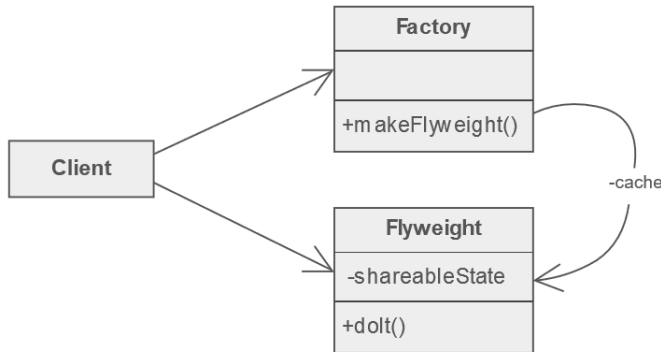
- Remark

- Use sharing to support a large number of objects that have part of their internal state in common where the other part of state can vary.
- Each "flyweight" object is divided into two pieces: state-dependent (extrinsic) part, and the state-independent (intrinsic) part.
 - Intrinsic state is stored (shared) in the Flyweight object.
 - Extrinsic state is stored or computed by client objects, and passed to the Flyweight when its operations are invoked.





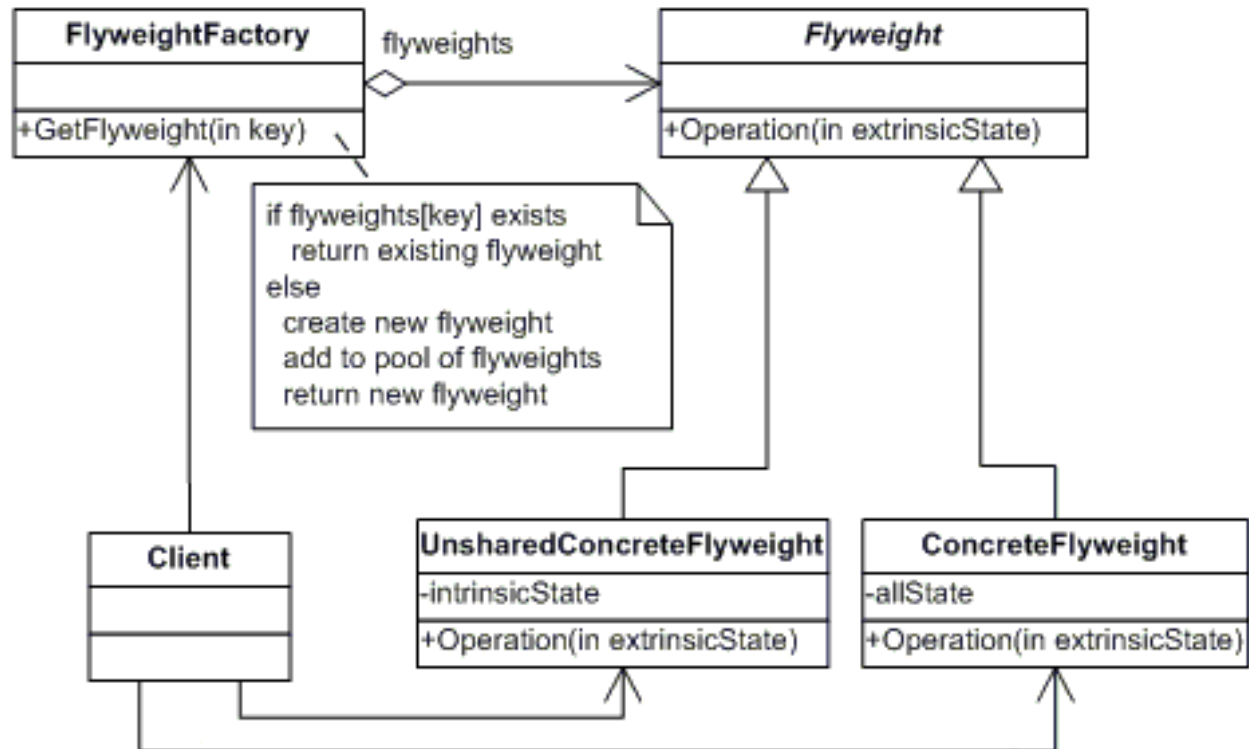
Flyweight vs Pool

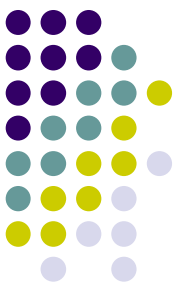


- *Differ How objects are used*
- **Flyweight** resources are immutable
 - Flyweight many clients can simultaneously use the same object.
- **Object pool** are mutable.
 - Object pool, at any given moment the object can be accessed only for one client
- Categorize the following:
 - Texture, Image, Sprite



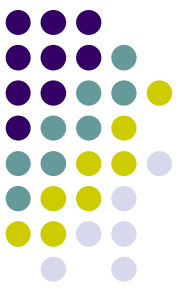
Flyweight





Flyweight

- Consequences
 - Flyweight pattern saves memory by sharing flyweight objects among clients. The amount of memory saved generally depends on the number of flyweight categories saved (for example a soldier category and a lieutenant category as discussed earlier).
- Related Patterns
 - Factory and Singleton patterns - Flyweights are usually created using a factory and the singleton is applied to that factory so that for each type or category of flyweights a single instance is returned.
 - State and Strategy Patterns - State and Strategy objects are usually implemented as Flyweights.

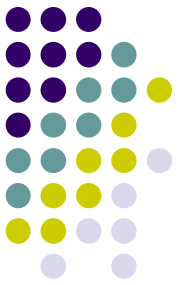


Design Patterns

- Easy tutorial book on design patterns
 - Get your copy today
 - Ecopy at Depaul Library through the safari link



Thank You!



- Questions?

